Research Notebook: Multi-Agent Learning and Liar's Dice

Research Notebook: Multi-Agent Learning and Liar’s Dice

Public Edition

Author: Mauricio Garcia Villanueva
Project: Liar’s Dice CFR Lab
Live demo: emgarviii.github.io/liars_dice/

Notebook Purpose

This notebook is a cleaned public version of the planning notes behind the project. The original notebook was exploratory. It had definitions, copied reminders, rough implementation thoughts, and places where my understanding changed over time. This edition keeps the research trajectory but removes clutter and makes the reasoning easier to follow.

The main arc is:

Start with imperfect-information games and multi-agent learning.

Study poker AI and CFR algorithms.

Try to apply those ideas to Liar’s Dice.

Realize that full classic Liar’s Dice is too large for a clean first solver.

Narrow the project into a one-claim challenge abstraction.

Build, evaluate, and publish the result.

Initial Motivation

I wanted a research project that connected game theory, machine learning, and strategic decision making. Poker AI was the obvious reference point, but I did not want to only summarize poker systems. I wanted to build something with hidden information that I could explain visually and eventually publish.

Liar’s Dice stood out because it is easy to understand and still strategically rich. Each player has private dice, and the public claim creates a decision under uncertainty. The game naturally raises the questions I was interested in:

How does an AI reason when it cannot see the full state?

How does self-play help a strategy improve?

What does equilibrium mean in a game with hidden information?

How do we evaluate whether the policy is actually good?

Concepts I Needed to Learn

The first concept was the information set. In a perfect-information game, the player can see the full board. In an imperfect-information game, the player sees only part of the state. The policy must therefore be based on what the player knows, not on the true full state.

The second concept was Nash equilibrium. A Nash equilibrium is a stable set of strategies where no player can improve by changing strategy alone. I originally treated equilibrium as a stronger and more automatic claim than I should have. The public version of the project is more careful. It talks about sampled CFR+ training and best-response diagnostics, but it does not claim that the published policy is proven to be at equilibrium.

The third concept was CFR. Counterfactual Regret Minimization repeatedly updates action probabilities by asking which actions would have performed better at an information set. CFR+ modifies that process by clipping negative regret, which can improve convergence behavior in practice.

The fourth concept was exploitability. A policy can beat simple opponents but still be weak against a targeted opponent. This became one of the most important lessons of the project.

Early Direction

The early project vision was broader than the final public demo. I wanted to build something close to classic Liar's Dice, where players raise claims until someone calls. That version is more familiar and more realistic.

The problem was the game tree. Once repeated raises and histories are included, the number of information sets grows quickly. A clean solver would need more careful abstraction, stronger training, and more evaluation. That was possible as future work, but it was too much for the first publishable version.

This is where the project changed from "solve Liar's Dice" to "build a tractable imperfect-information lab inspired by Liar's Dice." That was a better engineering decision because it created a version I could actually validate and explain.

The Simplified Game

The final public research mode uses one claim and one response.

The claimant says something like "two of face six." The claim refers to both players' dice combined. The responder sees only their own dice and must choose Believe or Challenge.

This keeps the hidden-information structure:

the claimant does not know the responder's dice,

the responder does not know the claimant's dice,

the claim may be true, false, or uncertain from one player's view.

It removes the classic raise loop. That is a real limitation, but it also makes the project explainable enough for a public portfolio demo.

Implementation Notes

The implementation split the project into two layers.

The Python layer is the research pipeline. It defines the simplified rules, generates game states, trains policies, validates distributions, simulates matches, and exports JSON artifacts.

The TypeScript layer is the public interface. It loads the exported policy and metrics files, renders the playable game, explains the decision guide, and exposes the method, results, classic comparison, and paper archive pages.

This split ended up being one of the best architectural decisions. It means GitHub Pages can host the project without a backend. The AI policy is frozen and reproducible rather than retrained in the browser.

Policy Key Lesson

An early weakness was that the policy did not condition strongly enough on changing dice counts. A strategy that makes sense at five dice each may not make sense later in the match when one side has fewer dice.

The updated public policy is remaining-dice-aware. Its keys include public dice counts and private hands. That improved benchmark performance because the policy could adapt to match state instead of reusing a fixed starting-state strategy.

This was an important engineering lesson: the representation of state is not just a detail. It directly controls what the policy can learn.

Evaluation Notes

At first, it was tempting to judge the AI by whether it felt smart in the browser. That was not enough. A playable demo is useful for communication, but evaluation needs repeatable metrics.

The benchmark suite compares the policy against several opponent profiles:

random behavior,

skeptical responses,

threshold-based responses,

truth-biased claims.

The updated policy performs much better across these profiles than the baseline. The benchmark average rose from 45.3 percent to 73.0 percent.

The best-response-style diagnostic made the story more honest. It showed that the policy still has exploitable pressure, especially when a stronger responder targets the claimant policy. That result changed how the project should be presented. The correct claim is not “the AI is optimal.” The correct claim is “the AI improved against benchmarks, and the evaluation exposed where it remains weak.”

Website Notes

The original project was not easy enough to show publicly. It had papers and code, but not a clean path for someone to understand the work quickly.

The public website solves that by giving different readers different depths:

The homepage explains the project and lets people play.

The method page explains the research engineering pipeline.

The results page explains the metrics and limitations.

The classic page shows the familiar Liar's Dice loop for comparison.

The papers page preserves the original course documents and adds polished public editions.

This matters because portfolio projects need more than implementation. They need a clear story.

What I Would Improve Next

The biggest technical improvement would be to train directly against stronger best-response pressure. That would move the project from benchmark improvement toward robustness.

The second improvement would be to extend the solver toward the classic raise-and-challenge game. That would require a larger state representation and more careful abstraction.

The third improvement would be better convergence reporting. The current checkpoints are useful, but a stronger research version would show more formal exploitability trends across training.

The fourth improvement would be richer visualization. The site could show policy behavior by information set after a round is revealed, while still avoiding hidden-hand leakage during play.

Final Reflection

The most useful part of the project was learning to separate ambition from evidence. It is easy to say "CFR finds equilibrium" in a general discussion. It is harder, and more valuable, to say exactly what my code trained, exactly what my tests measured, and exactly where the result still falls short.

That is why the public artifact is stronger than the original course submission. It does not just preserve the work. It makes the work inspectable.

The project now says what I wanted it to say: I researched imperfect information and multi-agent learning, built a working game-solving demo, evaluated it, and published the limitations clearly.